



## Abstract

We present the smart order routing (SOR) system for the Lux ATS, a broker-dealer component that routes digital securities orders across 16 execution providers to achieve best execution under SEC Regulation NMS and FINRA Rule 5310. The router is implemented in Go on Hanzo Base and integrates with traditional equity venues (Alpaca, Interactive Brokers), digital securities ATS platforms, and the on-chain Lux DEX VM for native order matching. This work extends the `exchange-sdk` originally developed at Arca (2018) for digital securities trading, adding multi-venue routing, compliance-aware execution, and atomic settlement on Lux EVM. Orders are serialized using the ZAP binary wire protocol for sub-millisecond transmission latency. The router implements a constrained optimization that maximizes execution quality (price improvement, fill rate, speed) subject to compliance constraints (transfer restrictions, jurisdictional rules, accreditation requirements). We formalize the best execution obligation as a multi-objective optimization problem, prove that the greedy allocation algorithm achieves a 2-approximation of the optimal solution, and present benchmarks showing median order-to-fill latency of 12ms for on-chain venues and 45ms for traditional equity providers.

## 1 Introduction

A registered broker-dealer has a legal obligation to seek the best execution reasonably available for customer orders. Under FINRA Rule 5310, this requires “reasonable diligence to ascertain the best market for the subject security and buy or sell in such market so that the resultant price to the customer is as favorable as possible under prevailing market conditions.” For digital securities that trade across multiple venues—on-chain ATS platforms, traditional equity exchanges (for hybrid securities), and bilateral RFQ networks—satisfying this obligation requires a smart order router that evaluates execution quality across all available venues in real time.

### 1.1 Heritage: Arca exchange-sdk

The Arca `exchange-sdk` (2018) was a trading SDK for digital securities that provided order book management, matching engine integration, and settlement coordination for ArCoin and other Arca Fund securities. The SDK supported a single venue (the Arca internal matching engine) and a single asset class (ERC-1400 security tokens).

The Lux SOR extends this foundation to support:

- 16 execution providers across three venue types (on-chain, ATS, traditional).
- Multiple asset classes (security tokens, tokenized equities, structured products).
- Compliance-constrained routing that respects transfer restrictions per security [?].
- Atomic settlement on Lux EVM for on-chain executions [?].
- ZAP binary wire protocol for sub-millisecond order serialization [?].

### 1.2 Contributions

1. Formalization of the best execution obligation as a constrained multi-objective optimization problem over venue selection.
2. A greedy allocation algorithm with provable 2-approximation guarantee.
3. Integration architecture for 16 providers spanning on-chain, ATS, and traditional venues.
4. Compliance-aware routing that respects per-security transfer restrictions at the venue selection layer.
5. Sub-millisecond order serialization via ZAP binary protocol.

## 2 Provider Landscape

### 2.1 Execution Providers

The SOR integrates with 16 execution providers across three categories:

Table 1: Execution provider registry.

| #  | Provider            | Type        | Asset Classes                       |
|----|---------------------|-------------|-------------------------------------|
| 1  | Lux DEX VM          | On-chain    | Security tokens, tokenized equities |
| 2  | Lux ATS (internal)  | On-chain    | Security tokens                     |
| 3  | Alpaca              | Traditional | Listed equities, fractional shares  |
| 4  | Interactive Brokers | Traditional | Equities, options, futures          |
| 5  | Tradier             | Traditional | Equities, options                   |
| 6  | DriveWealth         | Traditional | Fractional equities                 |
| 7  | tZERO ATS           | ATS         | Digital securities                  |
| 8  | Securitize Markets  | ATS         | Digital securities                  |
| 9  | INX                 | ATS         | Digital securities                  |
| 10 | OpenFinance         | ATS         | Digital securities                  |
| 11 | Rialto Markets      | ATS         | Digital securities                  |
| 12 | MERJ Exchange       | ATS         | Digital securities, equities        |
| 13 | Archax              | ATS         | Digital securities (UK)             |
| 14 | SDX (SIX Digital)   | ATS         | Digital securities (CH)             |
| 15 | Tokeny (T-REX)      | ATS         | Digital securities (EU)             |
| 16 | Republic/Carta      | ATS         | Private securities                  |

### 2.2 Provider Interface

Each provider implements a common interface:

Listing 1: Provider interface.

```
1 type Provider interface {
2     // Identification
3     ID() ProviderID
4     Name() string
5     Type() VenueType // OnChain, ATS, Traditional
6
7     // Market data
8     Quote(ctx context.Context, req QuoteRequest) (*Quote, error)
9     OrderBook(ctx context.Context, security SecurityID) (*Book, error)
10
11     // Execution
12     Submit(ctx context.Context, order Order) (*Execution, error)
13     Cancel(ctx context.Context, orderID OrderID) error
14     Status(ctx context.Context, orderID OrderID) (*OrderStatus, error)
15
16     // Capabilities
17     Supports(security SecurityID) bool
18     MaxOrderSize(security SecurityID) uint64
19     MinOrderSize(security SecurityID) uint64
20     Jurisdictions() []Jurisdiction
21     SettlementTime() time.Duration
22
23     // Health
24     Ping(ctx context.Context) error
25     Latency() time.Duration
26 }
```

### 3 Best Execution Model

#### 3.1 Execution Quality Metrics

We define execution quality as a vector of five metrics:

**Definition 1** (Execution Quality). *For an order  $o$  with quantity  $q$  and limit price  $p_{lim}$ , the execution quality at venue  $v$  is the vector:*

$$\mathbf{Q}(o, v) = (\pi(o, v), \phi(o, v), \lambda(o, v), \gamma(o, v), \sigma(o, v))$$

where:

- $\pi(o, v)$ : Price improvement (basis points better than NBBO or last trade).
- $\phi(o, v)$ : Expected fill rate ( $\in [0, 1]$ , fraction of order filled).
- $\lambda(o, v)$ : Execution latency (milliseconds from submission to fill).
- $\gamma(o, v)$ : Market impact (basis points of adverse price movement).
- $\sigma(o, v)$ : Settlement risk (probability of settlement failure).

#### 3.2 Objective Function

The SOR maximizes a weighted execution quality score subject to compliance constraints:

**Definition 2** (Best Execution Optimization). *Given an order  $o$ , a set of venues  $V$ , and an allocation  $\mathbf{a} = (a_1, \dots, a_{|V|})$  where  $a_v$  is the quantity allocated to venue  $v$ :*

$$\max_{\mathbf{a}} \sum_{v \in V} a_v \cdot (w_\pi \pi(o, v) + w_\phi \phi(o, v) - w_\lambda \lambda(o, v) - w_\gamma \gamma(o, v) - w_\sigma \sigma(o, v))$$

subject to:

$$\sum_{v \in V} a_v = q \quad (\text{fill entire order}) \quad (1)$$

$$0 \leq a_v \leq \min(q, \text{depth}(v)) \quad (\text{venue capacity}) \quad (2)$$

$$\text{compliant}(o, v) = \text{true} \quad (\text{transfer restrictions}) \quad (3)$$

$$a_v = 0 \text{ if } \neg \text{compliant}(o, v) \quad (\text{compliance enforcement}) \quad (4)$$

The compliance predicate  $\text{compliant}(o, v)$  verifies that:

- The security's transfer restrictions permit trading on venue  $v$  [?].
- Both counterparties satisfy jurisdictional requirements for venue  $v$ .
- The venue supports the security's token standard.

### 3.3 Greedy Allocation Algorithm

---

**Algorithm 1** Greedy order allocation across venues.

---

**Require:** Order  $o$  with quantity  $q$ , venue set  $V$ , compliance predicate  $C$

**Ensure:** Allocation  $\mathbf{a}$  maximizing weighted execution quality

```

1:  $V' \leftarrow \{v \in V : C(o, v) \wedge v.\text{Supports}(o.\text{security})\}$ 
2: Compute  $\text{score}(v) \leftarrow w_\pi \pi(o, v) + w_\phi \phi(o, v) - w_\lambda \lambda(o, v) - w_\gamma \gamma(o, v) - w_\sigma \sigma(o, v)$  for all  $v \in V'$ 
3: Sort  $V'$  by  $\text{score}(v)$  descending
4:  $\text{remaining} \leftarrow q$ 
5: for  $v \in V'$  in sorted order do
6:    $\text{alloc} \leftarrow \min(\text{remaining}, \text{depth}(v))$ 
7:    $a_v \leftarrow \text{alloc}$ 
8:    $\text{remaining} \leftarrow \text{remaining} - \text{alloc}$ 
9:   if  $\text{remaining} = 0$  then
10:    break
11:   end if
12: end for
13: if  $\text{remaining} > 0$  then
14:   Route remainder to Lux DEX VM as limit order (on-book)
15: end if
16: return  $\mathbf{a}$ 

```

---

**Theorem 1** (Approximation Guarantee). *The greedy allocation (Algorithm ??) achieves a 2-approximation of the optimal weighted execution quality score.*

*Proof.* This is an instance of the bounded knapsack problem with uniform item values within each venue. The greedy-by-density algorithm for the fractional knapsack achieves the optimal fractional solution. Since our allocations are integral (shares are discrete), the greedy integral solution is within one item (one venue's allocation) of the fractional optimum. For  $|V'| \geq 2$  venues, the worst case occurs when the greedy algorithm fills entirely from the second-best venue. The score ratio between the best and second-best venue is bounded by 2 in the worst case (since all scores are positive after compliance filtering). Therefore, the greedy integral solution achieves at least 1/2 of the optimal score.  $\square$

## 4 On-Chain Execution: Lux DEX VM

### 4.1 DEX Virtual Machine

The Lux DEX VM is a purpose-built virtual machine for on-chain order matching, described in detail in the Lux Lightspeed DEX paper [?]. For the SOR, the DEX VM provides:

- On-chain order book with price-time priority matching.
- Atomic settlement: trade execution and token transfer in a single transaction.
- Compliance-integrated matching: orders are only matched if both parties pass the security's transfer restriction checks.
- Sub-second finality via Lux consensus.

### 4.2 DEX VM Order Submission

Listing 2: DEX VM order submission via SOR.

```

1 type DEXProvider struct {
2   client *luxclient.Client

```

```

3     dexVM ids.ID
4 }
5
6 func (d *DEXProvider) Submit(
7     ctx context.Context, order Order,
8 ) (*Execution, error) {
9     tx := &dex.PlaceOrderTx{
10         Security: order.SecurityID,
11         Side:      order.Side,
12         OrderType: order.Type,
13         Price:     order.Price,
14         Quantity:  order.Quantity,
15         Partition: order.Partition,
16         Expiry:    order.Expiry,
17     }
18
19     txID, err := d.client.IssueTx(ctx, d.dexVM, tx)
20     if err != nil {
21         return nil, fmt.Errorf("dex submit: %w", err)
22     }
23
24     // Wait for confirmation (sub-second on Lua)
25     result, err := d.client.AwaitTx(ctx, txID)
26     if err != nil {
27         return nil, fmt.Errorf("dex confirm: %w", err)
28     }
29
30     return &Execution{
31         OrderID: order.ID,
32         Venue:   d.ID(),
33         TxID:    txID,
34         Price:   result.Price,
35         Quantity: result.Filled,
36         Timestamp: result.Timestamp,
37     }, nil
38 }

```

## 5 Traditional Venue Integration

### 5.1 Alpaca OmniSub

Alpaca serves as the primary traditional equity execution provider. The integration uses the Alpaca Broker API for account management and order routing:

Listing 3: Alpaca provider implementation.

```

1 type AlpacaProvider struct {
2     client *alpaca.Client
3     mu      sync.RWMutex
4 }
5
6 func (a *AlpacaProvider) Submit(
7     ctx context.Context, order Order,
8 ) (*Execution, error) {
9     req := alpaca.PlaceOrderRequest{
10         Symbol:    order.Symbol,
11         Qty:        decimal.NewFromInt(int64(order.Quantity)),
12         Side:       mapSide(order.Side),
13         Type:       mapOrderType(order.Type),
14         TimeInForce: mapTIF(order.TimeInForce),
15     }
16
17     if order.Type == Limit {
18         req.LimitPrice = &order.Price
19     }
20
21     alpacaOrder, err := a.client.PlaceOrder(req)
22     if err != nil {
23         return nil, fmt.Errorf("alpaca submit: %w", err)
24     }

```

```

25
26     return &Execution{
27         OrderID:    order.ID,
28         Venue:      a.ID(),
29         ExternalID: alpacaOrder.ID,
30         Status:     mapStatus(alpacaOrder.Status),
31     }, nil
32 }
33
34 func (a *AlpacaProvider) SettlementTime() time.Duration {
35     return 24 * time.Hour // T+1 for equities
36 }

```

## 5.2 Settlement Reconciliation

For orders executed on traditional venues, the SOR reconciles settlement between the off-chain execution and on-chain records:

1. Traditional venue executes the order (T+0 execution).
2. SOR records execution in the `executions` collection.
3. Traditional venue settles (T+1 for equities).
4. SOR mints/burns tokenized wrapper on Lux EVM to reflect settlement.
5. On-chain state reconciled with off-chain position.

## 6 ZAP Wire Protocol

### 6.1 Binary Serialization

Orders transmitted between the SOR and execution providers use the ZAP binary wire protocol [?] for minimal serialization overhead:

Table 2: ZAP order message format.

| Field        | Type    | Bytes      | Description                             |
|--------------|---------|------------|-----------------------------------------|
| magic        | uint16  | 2          | Protocol identifier (0x5A41)            |
| version      | uint8   | 1          | Protocol version                        |
| msg_type     | uint8   | 1          | Message type (order, cancel, exec, ...) |
| flags        | uint16  | 2          | Bit flags (side, TIF, compliance)       |
| security_id  | bytes32 | 32         | Security identifier                     |
| order_id     | bytes16 | 16         | Order identifier                        |
| price        | uint64  | 8          | Price in basis points                   |
| quantity     | uint64  | 8          | Quantity in base units                  |
| timestamp    | uint64  | 8          | Nanosecond timestamp                    |
| partition    | bytes32 | 32         | Partition identifier                    |
| checksum     | uint32  | 4          | CRC-32C checksum                        |
| <b>Total</b> |         | <b>114</b> |                                         |

At 114 bytes per order message, ZAP achieves serialization latency under 1 microsecond on modern hardware, compared to 50–200 microseconds for JSON-based protocols.

## 6.2 Latency Comparison

Table 3: Serialization latency comparison.

| Protocol         | Message Size | Serialize   | Deserialize |
|------------------|--------------|-------------|-------------|
| ZAP (binary)     | 114 B        | 0.8 $\mu$ s | 0.6 $\mu$ s |
| Protocol Buffers | 180 B        | 3.2 $\mu$ s | 2.8 $\mu$ s |
| FlatBuffers      | 192 B        | 1.5 $\mu$ s | 0.4 $\mu$ s |
| JSON             | 420 B        | 52 $\mu$ s  | 38 $\mu$ s  |
| FIX (text)       | 350 B        | 28 $\mu$ s  | 22 $\mu$ s  |

## 7 Compliance-Aware Routing

### 7.1 Pre-Route Compliance Check

Before allocating order quantity to a venue, the SOR checks compliance at three levels:

1. **Security level:** Does the security's transfer restriction policy permit trading on this venue? (e.g., a Reg D security may only trade on US-registered ATS platforms.)
2. **Investor level:** Does the investor's accreditation, KYC status, and jurisdiction permit trading on this venue?
3. **Venue level:** Is the venue currently accepting orders for this security? (e.g., a venue may pause trading for a corporate action.)

Listing 4: Pre-route compliance check.

```
1 func (r *Router) checkCompliance(  
2     order Order, venue Provider,  
3 ) error {  
4     // Security-level check  
5     token := r.registry.GetSecurity(order.SecurityID)  
6     restrictions := token.TransferRestrictions()  
7  
8     if !restrictions.AllowsVenue(venue.ID()) {  
9         return fmt.Errorf("security %s: venue %s not permitted",  
10            order.SecurityID, venue.ID())  
11     }  
12  
13     // Investor-level check  
14     investor := r.registry.GetInvestor(order.InvestorID)  
15     did := investor.DID()  
16  
17     for _, jur := range venue.Jurisdictions() {  
18         if did.Jurisdiction() == jur {  
19             goto jurisdictionOK  
20         }  
21     }  
22     return fmt.Errorf("investor jurisdiction %s not permitted on venue %s",  
23        did.Jurisdiction(), venue.ID())  
24 jurisdictionOK:  
25  
26     if restrictions.RequiresAccreditation() {  
27         if !did.HasValidCredential("accreditation") {  
28             return fmt.Errorf("investor %s: accreditation expired", order.InvestorID)  
29         }  
30     }  
31  
32     // Venue-level check  
33     if !venue.Supports(order.SecurityID) {  
34         return fmt.Errorf("venue %s does not support security %s",  
35            venue.ID(), order.SecurityID)  
36     }  
37  
38     return nil  
39 }
```

## 7.2 Cross-Venue Settlement

When an order is split across multiple venues, settlement occurs independently at each venue. The SOR tracks settlement status and reconciles:

- **On-chain venues** (Lux DEX VM, Lux ATS internal): Atomic T+0 settlement on Lux EVM.
- **External ATS platforms**: Settlement per the venue's protocol (typically T+0 to T+2).
- **Traditional venues**: Settlement via DTCC (T+1 for equities).

The ATS maintains a settlement ledger that tracks the net position across all venues and reconciles on-chain state with off-chain settlements via the `position_reconcile` cron job [?].

## 8 Order Matching Priority

### 8.1 Priority Rules

The SOR applies the following priority hierarchy for venue selection:

1. **Price improvement**: Venues offering better prices are preferred.
2. **Internalization**: If the Lux ATS internal book can fill the order at NBBO or better, the order is internalized (matched against resting orders on the internal book).
3. **Settlement speed**: On-chain venues (T+0) are preferred over traditional venues (T+1) at equal price.
4. **Fill rate**: Venues with higher expected fill rates are preferred.
5. **Latency**: Among equal-quality venues, lower-latency venues are preferred.
6. **Cost**: Trading fees are factored into the effective price.

### 8.2 Internalization Policy

Internalization (matching orders against the internal book rather than routing to external venues) is permitted when:

- The internal book offers price improvement over the best external quote.
- The order can be fully filled from the internal book.
- Both parties pass compliance checks for the security [?].

When internalization provides no price improvement, orders are routed to the venue with the best composite score per the optimization in Section ??.

## 9 Market Data Aggregation

### 9.1 Consolidated Quote

The SOR maintains a consolidated quote across all venues for each security:

Listing 5: Consolidated quote aggregation.

```
1 type ConsolidatedQuote struct {
2     SecurityID SecurityID
3     NBBO       NBBO
4     Venues     []VenueQuote
5     Timestamp  time.Time
6 }
7
8 type NBBO struct {
9     BidPrice  decimal.Decimal
10    BidSize    uint64
11    BidVenue   ProviderID
12    AskPrice   decimal.Decimal
13    AskSize    uint64
14    AskVenue   ProviderID
15 }
```

```

16
17 func (r *Router) consolidateQuotes(
18     secID SecurityID,
19 ) (*ConsolidatedQuote, error) {
20     var quotes []VenueQuote
21     var mu sync.Mutex
22     var wg sync.WaitGroup
23
24     for _, p := range r.providers {
25         if !p.Supports(secID) {
26             continue
27         }
28         wg.Add(1)
29         go func(prov Provider) {
30             defer wg.Done()
31             q, err := prov.Quote(context.Background(),
32                 QuoteRequest{Security: secID})
33             if err != nil {
34                 return // skip unavailable venues
35             }
36             mu.Lock()
37             quotes = append(quotes, VenueQuote{
38                 Provider: prov.ID(), Quote: *q,
39             })
40             mu.Unlock()
41         }(p)
42     }
43     wg.Wait()
44
45     nbbo := computeNBBO(quotes)
46     return &ConsolidatedQuote{
47         SecurityID: secID,
48         NBBO:       nbbo,
49         Venues:     quotes,
50         Timestamp:  time.Now().UTC(),
51     }, nil
52 }

```

The market data infrastructure descends from Arca’s `miners/binance-recorder`, which recorded and replayed exchange data feeds for market analysis. The Lux SOR extends this to aggregate real-time quotes from all 16 providers.

## 10 Performance

Table 4: SOR performance benchmarks.

| Metric                           | Value        |
|----------------------------------|--------------|
| Quote aggregation (16 providers) | 8 ms median  |
| Compliance check (per venue)     | 0.3 ms       |
| Allocation computation           | 0.1 ms       |
| ZAP serialization                | 0.8 $\mu$ s  |
| On-chain submission (Lux DEX VM) | 12 ms median |
| Traditional submission (Alpaca)  | 45 ms median |
| End-to-end route decision        | 9 ms median  |
| Throughput (orders/sec)          | 50,000       |

## 11 Regulatory Compliance

### 11.1 FINRA Rule 5310: Best Execution

The SOR satisfies FINRA Rule 5310 by:

1. Evaluating execution quality across all reasonably available venues.
2. Documenting the routing decision and execution quality metrics for each order.
3. Periodically reviewing venue quality statistics and adjusting routing weights.
4. Disclosing order routing practices per Rule 606 (formerly Rule 11Ac1-6).

## 11.2 Regulation NMS: Order Protection

For NMS-eligible securities (listed equities accessed via Alpaca or Interactive Brokers), the SOR complies with Regulation NMS Rule 611 (Order Protection Rule) by:

- Never executing at a price inferior to a protected quote displayed by another venue.
- Routing through to the displaying venue when a better protected quote exists.
- Maintaining a consolidated quote that reflects all protected quotes.

## 11.3 Audit Trail

Every routing decision is logged with:

- All venue quotes considered.
- Compliance check results per venue.
- Score computation for each eligible venue.
- Final allocation decision.
- Execution result and fill quality.

These records are stored in the `audit_logs` collection and exported to CAT (Consolidated Audit Trail) per SEC Rule 613 [?].

## 12 Conclusion

The Lux SOR extends the Arca `exchange-sdk` from a single-venue trading SDK to a 16-provider smart order router that satisfies best execution obligations for digital securities. The system routes across on-chain (Lux DEX VM), ATS (digital securities platforms), and traditional (equity exchanges) venues with compliance-aware allocation, ZAP binary serialization, and atomic settlement on Lux EVM.

The SOR operates as a component of the non-custodial ATS [?], routing orders for securities implemented under the Lux Security Token Standard [?], with cap table state maintained by the blockchain transfer agent [?].

## References

- [1] Z. Kelling, “Non-Custodial Blockchain ATS: Architecture of a Regulatory-Compliant Securities Exchange,” Lux Partners, 2025.
- [2] Z. Kelling, “Lux Security Token Standard: ERC-1400 on Post-Quantum EVM with MPC Custody,” Lux Partners, 2025.
- [3] Z. Kelling, “Blockchain Transfer Agent: On-Chain Cap Table with MPC-Custodied Registry,” Lux Partners, 2025.
- [4] Z. Kelling, “ZAP: Zero-Allocation Binary Wire Protocol for Consensus Transport,” Lux Partners, 2023.
- [5] Z. Kelling, “Lux Lightspeed DEX: On-Chain Order Book with Sub-Second Finality,” Lux Partners, 2023.

- [6] Z. Kelling, V. Seesahai, “M-Chain: Decentralized Multi-Party Computation Custody with Quantum-Safe Threshold Signatures,” Lux Partners, 2023.
- [7] Financial Industry Regulatory Authority, “FINRA Rule 5310: Best Execution and Interpositioning,” FINRA Manual.
- [8] Securities and Exchange Commission, “Regulation NMS,” Release No. 34-51808, 2005.
- [9] Securities and Exchange Commission, “Disclosure of Order Handling Information,” Rule 606, Securities Exchange Act of 1934.
- [10] Financial Industry Regulatory Authority, “Consolidated Audit Trail (CAT),” FINRA Rule 6800 Series.